

PRACTICAL 12:

PROGRAM 1: Producer–Consumer Problem using Counting Semaphores and POSIX Threads

AIM

To implement the **Producer–Consumer problem** using **counting semaphores and POSIX threads**, and to understand synchronization between producer and consumer threads for different buffer sizes.

THEORY

The **Producer–Consumer problem** is a classical synchronization problem in Operating Systems.

- The **Producer** creates/produces data and places it into a shared buffer.
- The **Consumer** removes/consumes data from the buffer.
- Since both access the same buffer, synchronization is required.

Three synchronization mechanisms are commonly used:

Semaphore	Purpose
empty	Counts the number of empty spaces in the buffer
full	Counts the number of filled spaces in the buffer
mutex	Ensures only one thread accesses the buffer at a time

Important operations

`sem_wait()`

- Decreases the semaphore value.
- If the value is 0, the thread waits.

sem_post()

- Increases the semaphore value.
- It wakes a waiting thread if necessary.

POSIX Threads

POSIX threads, commonly called **pthread**s, allow multiple threads to execute concurrently within a process.

Functions used:

- `pthread_create()` → creates a thread
- `pthread_join()` → waits for a thread to finish
- `sem_init()` → initializes a semaphore
- `sem_wait()` → waits/decrements a semaphore
- `sem_post()` → signals/increments a semaphore
- `sem_destroy()` → destroys a semaphore

Program:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3  #include <pthread.h>
4  #include <semaphore.h>
5  #include <unistd.h>
6
7  #define BUFFER_SIZE 5
8  #define ITEMS 10
9
10 int buffer[BUFFER_SIZE];
11 int in = 0;
12 int out = 0;
13
14 sem_t empty;
15 sem_t full;
16 sem_t mutex;
17
18 void *producer(void *arg)
19 {
20     for (int i = 1; i <= ITEMS; i++)
21     {
22         sem_wait(&empty);
23         sem_wait(&mutex);
24
25         buffer[in] = i;
26         printf("Producer produced: %d\n", i);
27         in = (in + 1) % BUFFER_SIZE;
28
29         sem_post(&mutex);
30         sem_post(&full);
31
32         sleep(1);
33     }
34
35     return NULL;
36 }
37
38 void *consumer(void *arg)
39 {
40     for (int i = 1; i <= ITEMS; i++)
41     {
42         sem_wait(&full);
43         sem_wait(&mutex);
```

```

39  {
40
41      {
42          sem_wait(&full);
43          sem_wait(&mutex);
44
45          int item = buffer[out];
46          printf("Consumer consumed: %d\n", item);
47          out = (out + 1) % BUFFER_SIZE;
48
49          sem_post(&mutex);
50          sem_post(&empty);
51
52          sleep(1);
53      }
54
55      return NULL;
56  }
57
58  int main()
59  {
60      pthread_t producerThread, consumerThread;
61
62      sem_init(&empty, 0, BUFFER_SIZE);
63      sem_init(&full, 0, 0);
64      sem_init(&mutex, 0, 1);
65
66      pthread_create(&producerThread, NULL, producer, NULL);
67      pthread_create(&consumerThread, NULL, consumer, NULL);
68
69      pthread_join(producerThread, NULL);
70      pthread_join(consumerThread, NULL);
71
72      sem_destroy(&empty);
73      sem_destroy(&full);
74      sem_destroy(&mutex);
75
76      return 0;
77  }

```

OUTPUT:

```
PS C:\Users\panak\Downloads\OSSP PRACTICALS> gcc producer_consumer.c -o producer_consumer -pthread
PS C:\Users\panak\Downloads\OSSP PRACTICALS> ./producer_consumer
Producer produced: 1
Consumer consumed: 1
Producer produced: 2
Consumer consumed: 2
Producer produced: 3
Consumer consumed: 3
Producer produced: 4
Consumer consumed: 4
Producer produced: 5
Consumer consumed: 5
Producer produced: 6
Consumer consumed: 6
Producer produced: 7
Consumer consumed: 7
Producer produced: 8
Consumer consumed: 8
Producer produced: 9
Consumer consumed: 9
Producer produced: 10
Consumer consumed: 10
```

Result:

The Producer–Consumer problem was successfully implemented using POSIX threads and counting semaphores. The semaphores synchronized access to the shared buffer and prevented simultaneous access to the critical section.

PROGRAM 2: Deadlock and Deadlock Prevention using Resource Ordering

AIM

To design a **deadlock scenario involving multiple threads competing for shared resources**, identify the four necessary conditions for deadlock, and implement **deadlock prevention using resource ordering**.

THEORY

What is Deadlock?

A **deadlock** occurs when two or more processes/threads are permanently waiting for resources held by each other.

For example:

Thread 1 → holds Resource A → waits for Resource B

Thread 2 → holds Resource B → waits for Resource A

Neither thread can continue.

Four Necessary Conditions for Deadlock

Deadlock can occur only when all four of these conditions are present:

1. Mutual Exclusion

Only one process/thread can use a resource at a time.

2. Hold and Wait

A thread holds one resource while waiting for another resource.

3. No Preemption

A resource cannot be forcibly taken away from a thread.

4. Circular Wait

A circular chain exists where each thread is waiting for a resource held by another thread.

Example:

T1 → waiting for R2
↑ ↓
R1 ← T2

DEADLOCK PREVENTION — RESOURCE ORDERING

One way to prevent deadlock is **resource ordering**.

We assign an order to resources:

Resource 1 < Resource 2

Every thread must acquire resources in the **same order**.

For example:

Correct:

Thread 1: Resource 1 → Resource 2

Thread 2: Resource 1 → Resource 2

This prevents **circular wait**, because no thread can acquire Resource 2 and then go backward and wait for Resource 1.

PROGRAM:

```
1  #include <stdio.h>
2  #include <pthread.h>
3  #include <unistd.h>
4
5  pthread_mutex_t resource1;
6  pthread_mutex_t resource2;
7
8  void *thread1(void *arg)
9  {
10     printf("Thread 1: Waiting for Resource 1\n");
11     pthread_mutex_lock(&resource1);
12
13     printf("Thread 1: Acquired Resource 1\n");
14
15     sleep(1);
16
17     printf("Thread 1: Waiting for Resource 2\n");
18     pthread_mutex_lock(&resource2);
19
20     printf("Thread 1: Acquired Resource 2\n");
21     printf("Thread 1: Using both resources\n");
22
23     pthread_mutex_unlock(&resource2);
24     pthread_mutex_unlock(&resource1);
25
26     printf("Thread 1: Released both resources\n");
27
28     return NULL;
29 }
30
31 void *thread2(void *arg)
32 {
33     printf("Thread 2: Waiting for Resource 1\n");
34     pthread_mutex_lock(&resource1);
35
36     printf("Thread 2: Acquired Resource 1\n");
37
38     sleep(1);
39
40     printf("Thread 2: Waiting for Resource 2\n");
41     pthread_mutex_lock(&resource2);
42
```

```
42
43     printf("Thread 2: Acquired Resource 2\n");
44     printf("Thread 2: Using both resources\n");
45
46     pthread_mutex_unlock(&resource2);
47     pthread_mutex_unlock(&resource1);
48
49     printf("Thread 2: Released both resources\n");
50
51     return NULL;
52 }
53
54 int main()
55 {
56     pthread_t t1, t2;
57
58     pthread_mutex_init(&resource1, NULL);
59     pthread_mutex_init(&resource2, NULL);
60
61     pthread_create(&t1, NULL, thread1, NULL);
62     pthread_create(&t2, NULL, thread2, NULL);
63
64     pthread_join(t1, NULL);
65     pthread_join(t2, NULL);
66
67     pthread_mutex_destroy(&resource1);
68     pthread_mutex_destroy(&resource2);
69
70     printf("Deadlock prevented using resource ordering.\n");
71
72     return 0;
73 }
```

OUTPUT:

```
● PS C:\Users\panak\Downloads\OSSP PRACTICALS> gcc deadlock_prevention.c -o deadlock_prevention -pthread
● PS C:\Users\panak\Downloads\OSSP PRACTICALS> ./deadlock_prevention
Thread 1: Waiting for Resource 1
Thread 1: Acquired Resource 1
Thread 2: Waiting for Resource 1
Thread 1: Waiting for Resource 2
Thread 1: Acquired Resource 2
Thread 1: Using both resources
Thread 1: Released both resources
Thread 2: Acquired Resource 1
Thread 2: Waiting for Resource 2
Thread 2: Acquired Resource 2
Thread 2: Using both resources
Thread 2: Released both resources
Deadlock prevented using resource ordering.
○ PS C:\Users\panak\Downloads\OSSP PRACTICALS> █
```

CONCLUSION

The Producer–Consumer problem was successfully implemented using **POSIX threads and counting semaphores**. The empty, full, and mutex semaphores synchronized access to the shared buffer and prevented race conditions.

The deadlock concept was studied using multiple threads and shared resources. The **four necessary conditions of deadlock** were identified, and **resource ordering** was implemented to prevent circular wait and avoid deadlock.